

SIMATS ENGINEERING
Co3-ASSESSMENT TOOL 2
Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Dr.V.Parthipan

QUESTIONS:4

Total Marks:40

Code of Conduct:

*I **C.BHUVANESH (192521108)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:*c.bhuvanesh*

Criteria	Understanding the case 2.5	Application of Concepts 2.5	Analysis 2	Solution Design 2	Clarity 1	Total (10)
Q1						
Q2						
Q3						
Q4						

Faculty Signature

QUESTIONS:4**Total Marks:40**

1. A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss**, **180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

1. Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.
2. Justify your answer based on the communication requirements of real-time multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

The problem is mainly related to the **transport protocol and its interaction with the application layer**. For real-time video conferencing, using **TCP** can cause delays because TCP retransmits lost packets and waits for acknowledgements. With **8% packet loss** and **180 ms jitter**, these retransmissions can increase delay and cause frozen video and distorted audio.

However, the **application layer also plays an important role**. If the conferencing application uses a large buffer, it may wait too long for missing packets, increasing communication delay. If the buffer is too small, packet loss and jitter may directly affect audio and video quality.

2. Why real-time multimedia applications are different

Video conferencing requires **low delay, low jitter, and continuous delivery** rather than perfect delivery of every packet.

- **TCP** provides reliable delivery, but retransmission and congestion control can introduce delays.
- **UDP** is commonly preferred for real-time audio and video because it does not wait for lost packets to be retransmitted.
- A small amount of packet loss is often better than waiting for delayed packets.

- **180 ms jitter** can cause audio/video synchronization problems.
- **8% packet loss** is high enough to cause noticeable voice distortion and frozen frames.
- Fluctuating bandwidth can make the problem worse if the application does not adapt its video quality.

Therefore, **UDP with suitable application-level mechanisms** is generally more appropriate for real-time conferencing.

3. Recommended improvements

Protocol-level improvements

1. Use **UDP-based real-time transport** instead of TCP where appropriate.
2. Use **RTP/RTCP** over UDP for audio/video delivery and quality monitoring.
3. Apply **congestion control** to reduce traffic during peak periods.
4. Use **packet-loss recovery techniques**, such as Forward Error Correction (FEC), when suitable.
5. Use **QoS (Quality of Service)** to prioritize voice and video traffic over less time-sensitive traffic.

Application-level improvements

1. Use an **adaptive jitter buffer** to handle variations in packet arrival time.
2. Use **adaptive bitrate (ABR)** so video resolution and bitrate automatically decrease when bandwidth becomes limited.
3. Prioritize **audio over video** because audio is more important for maintaining communication.
4. Use **packet-loss concealment** to reduce the effect of missing audio/video packets.
5. Avoid excessively large buffers because they can create additional delay.
6. Monitor network conditions continuously and dynamically adjust video quality.

Conclusion

The **8% packet loss, 180 ms jitter, and fluctuating bandwidth** strongly indicate that the conferencing system is experiencing network and transport-related problems, but the application's buffering and adaptation strategy can significantly affect the final user experience. For real-time conferencing, **UDP-based transport combined with RTP/RTCP, adaptive buffering, adaptive bitrate, QoS, and**

packet-loss recovery would provide better performance than relying on TCP's retransmission-based reliability.

2. A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the **average DNS lookup time is approximately 800 ms**, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as **Anycast DNS**, **DNS pre-fetching**, and **TTL optimization** to reduce the lookup time below **50 ms**.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

1. Reasons for the high DNS resolution time

The **800 ms average DNS lookup time** is very high and can be caused by several factors:

- **Long geographical distance:** Users may be contacting DNS servers located far away.
- **DNS recursion delays:** A recursive DNS server may need to contact multiple DNS servers before finding the required IP address.
- **High DNS server load:** Heavy traffic can increase processing and response time.
- **Poor DNS caching:** If DNS records are not cached effectively, repeated queries require fresh resolution.
- **High network latency:** Congestion and unstable network connections can delay DNS requests.
- **Improper TTL values:** Very short TTL values cause DNS records to expire frequently, increasing the number of DNS lookups.

- **Single-server dependency:** If one DNS server is overloaded or unavailable, users may experience additional delays.
-

2. Proposed DNS architecture

The company can use a **distributed Anycast DNS architecture combined with DNS caching, pre-fetching, and optimized TTL values.**

Architecture:

User → Nearest Anycast DNS Server → Cached DNS Response → Web Server

5

a) Anycast DNS

Deploy DNS servers at multiple geographical locations such as **Asia, Europe, North America, and Australia.**

All DNS servers advertise the **same IP address** using routing protocols such as BGP. The network automatically directs users to a nearby or optimal DNS server.

Example:

- User in India → Asian DNS server
- User in Europe → European DNS server
- User in USA → North American DNS server

This reduces network round-trip time and improves availability.

b) DNS pre-fetching

Frequently accessed domain records can be **pre-fetched before their TTL expires.**

For example, if a DNS record has a TTL of 300 seconds, the DNS resolver can refresh it before expiration. This ensures that users usually receive the answer directly from the cache instead of waiting for a complete recursive lookup.

c) TTL optimization

Use suitable TTL values for different DNS records.

- Frequently changing records → shorter TTL
- Stable records → longer TTL

For example, stable web-service records could use a TTL of **300–3600 seconds**, depending on how quickly changes need to propagate.

Longer TTLs reduce repeated DNS queries and improve response time.

d) Multiple DNS servers and failover

Deploy several authoritative DNS servers across different regions. If one server fails, DNS traffic can automatically be redirected to another available server.

This prevents a single point of failure and provides **continuous service availability**.

Target performance

With users directed to nearby Anycast servers and frequently requested records available in cache, DNS responses can often be reduced to **well below 50 ms** for cached queries.

However, **50 ms cannot be guaranteed for every uncached recursive lookup**, because Internet distance and upstream DNS response times can vary.

3. Advantages and limitations

Feature	Advantages	Limitations
Anycast DNS	Reduces latency, improves availability, distributes traffic	More complex routing and management
DNS Pre-fetching	Keeps popular records ready in cache and reduces lookup delays	Uses additional DNS queries and resources
TTL Optimization	Reduces unnecessary DNS lookups and improves cache efficiency	Long TTL can delay DNS changes

Multiple DNS servers	Provides fault tolerance and continuous service	Higher infrastructure and maintenance cost
-----------------------------	---	--

Caching	Very fast responses for repeated requests	Cached records can become outdated
----------------	---	------------------------------------

Conclusion

The company's **800 ms DNS lookup time** is mainly caused by geographical latency, inefficient caching, overloaded DNS infrastructure, and repeated recursive lookups. A **globally distributed Anycast DNS architecture**, supported by **DNS pre-fetching, optimized TTL values, effective caching, and multiple redundant DNS servers**, can significantly reduce DNS response time and improve availability.

3.A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly **800 ms** because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

4. Examine the impact of centralized DNS architecture on application performance.
5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
6. Analyze how **TTL values, Anycast routing, and DNS caching** influence system performance and fault tolerance.

1. Impact of centralized DNS architecture on application performance

In the given situation, **all DNS requests are sent to a single primary DNS server**. This creates a major **performance bottleneck and single point of failure**.

The 800 ms DNS lookup time causes users to wait before their browsers can even connect to the cloud application. During promotional events, millions of users generate a large number of DNS queries, which can overload the server and increase response time.

The main problems are:

- **High latency:** Every user must contact the same DNS server, even when the server is geographically far away.
- **Server overload:** A large number of simultaneous queries can exceed the server's processing capacity.
- **Single point of failure:** If the primary DNS server fails, users may be unable to resolve the application domain.
- **Poor scalability:** Adding more users increases the load on the same server.
- **Increased application delay:** DNS resolution occurs before the user can establish a connection with the cloud service.

Therefore, the centralized architecture is unsuitable for a global service with millions of users.

2. Proposed distributed DNS solution

The company should implement a **globally distributed DNS architecture using Anycast routing, multiple authoritative DNS servers, and DNS caching.**

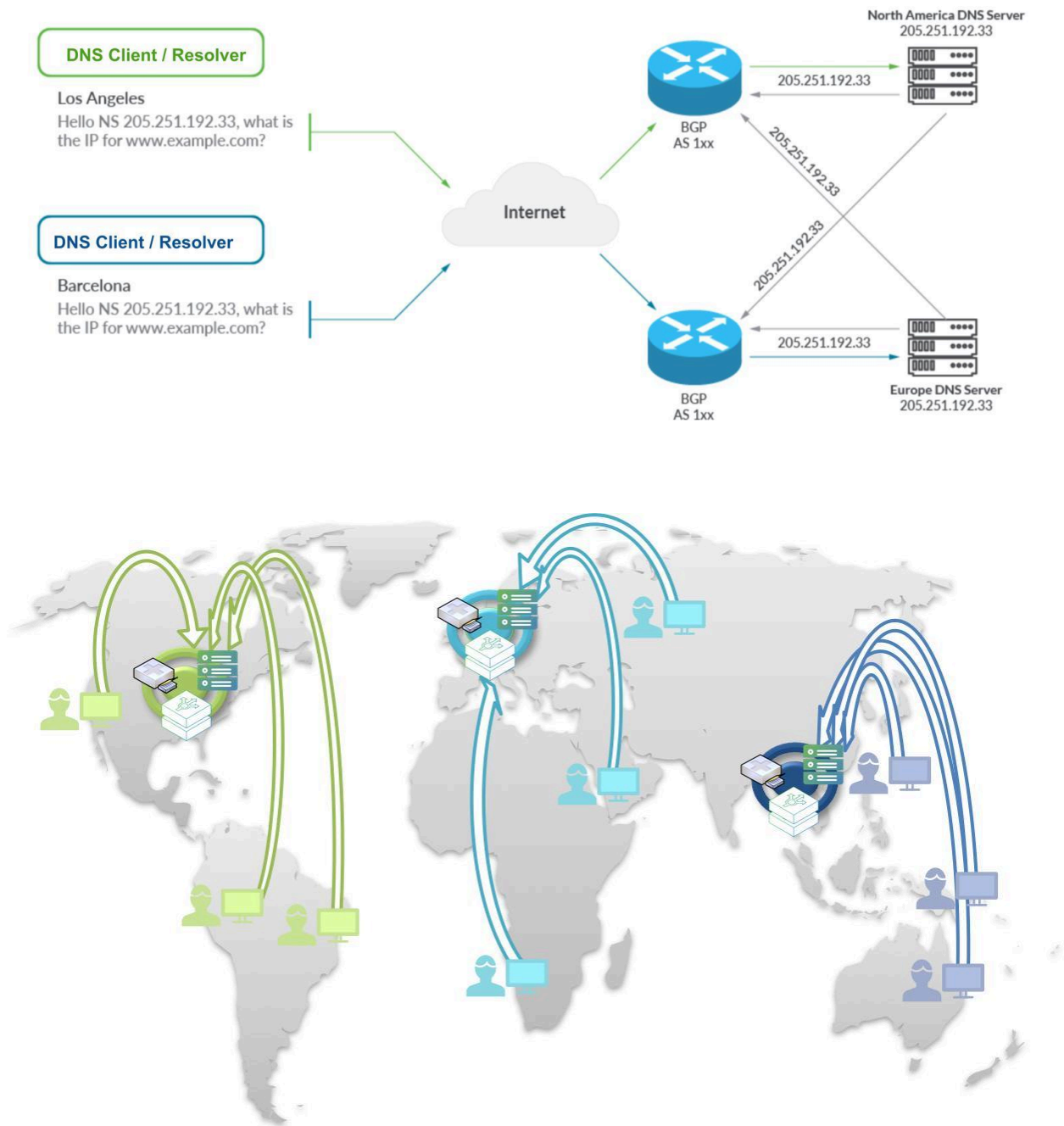
DNS Anycast

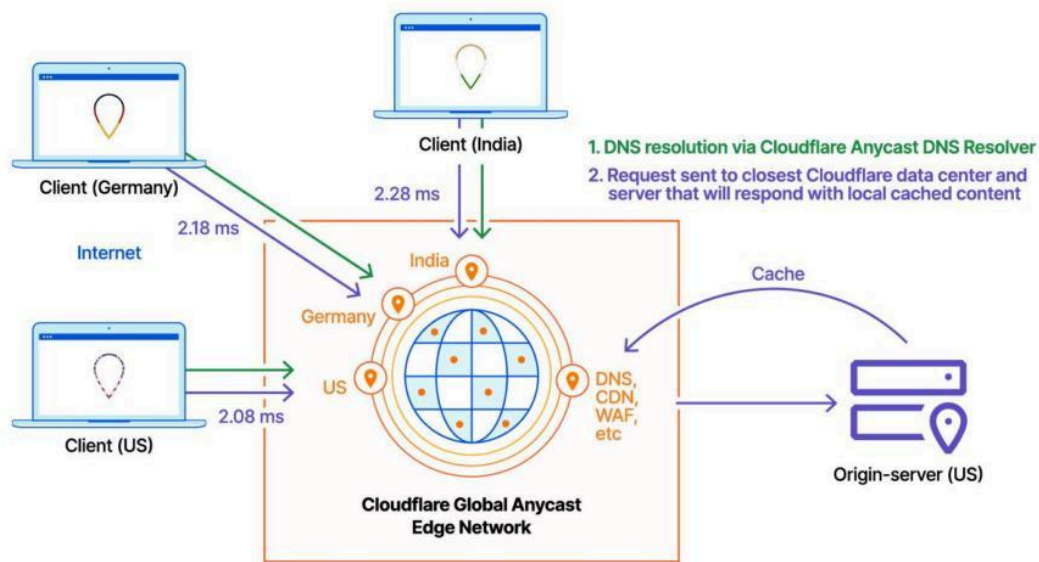
DNS Client / Resolver

Los Angeles
Hello NS 205.251.192.33, what is
the IP for www.example.com?

DNS Client / Resolver

Barcelona
Hello NS 205.251.192.33, what is
the IP for www.example.com?





6

Proposed architecture

Users worldwide → Nearest Anycast DNS node → DNS cache → Authoritative DNS server → Cloud application

Step 1: Deploy DNS servers globally

Place multiple DNS servers in different geographical regions:

- Asia
- Europe
- North America
- South America
- Australia

This allows users to obtain DNS responses from servers closer to their location.

Step 2: Use Anycast routing

All DNS servers can advertise the **same IP address** using Anycast routing.

For example:

- User in India → Asian DNS node
- User in Germany → European DNS node

- User in USA → North American DNS node

The routing system directs the query to an appropriate nearby/optimal DNS instance.

This significantly reduces network round-trip time.

Step 3: Use DNS caching

Frequently requested DNS records should be stored in caches.

For example, if thousands of users request the same cloud application's IP address, the DNS resolver can return the cached answer instead of performing a new lookup every time.

This reduces:

- DNS traffic
- Server workload
- Lookup latency

Step 4: Provide redundant DNS servers

Each region should have multiple DNS servers. If one server fails, traffic can be redirected to another available server.

This provides **high availability and fault tolerance**.

3. Effect of TTL, Anycast routing, and DNS caching

Technique	Effect on Performance	Effect on Fault Tolerance
TTL values	Appropriate TTL reduces repeated DNS lookups	Longer TTL can allow cached records to continue working during temporary DNS failures
Anycast routing	Sends users to nearby DNS servers, reducing latency	If one Anycast node fails, routing can direct traffic to another node

DNS caching	Provides fast responses without repeated recursive lookups	Cached records may remain usable even if the authoritative server is temporarily unavailable
--------------------	--	--

TTL values

TTL (Time To Live) determines how long a DNS record can remain in a cache.

- **Very short TTL:** Changes propagate quickly, but DNS queries increase.
- **Very long TTL:** Reduces DNS queries and improves performance, but changes take longer to propagate.
- **Balanced TTL:** Provides a good compromise between performance and flexibility.

For stable cloud services, a moderate TTL such as **300–3600 seconds** can be considered, depending on how frequently the records change.

Anycast routing

Anycast allows multiple DNS servers to use the same IP address. Internet routing directs users toward an appropriate DNS location.

Benefits:

- Lower latency
- Global traffic distribution
- Better scalability
- Automatic routing around failed nodes

DNS caching

Caching is one of the most important methods for reducing DNS load. Once a domain's IP address is cached, subsequent requests can be answered quickly without contacting the authoritative server.

For example:

Without caching:

User → DNS server → Recursive lookup → Authoritative server → Response

With caching:

User → Cache → Response

This can reduce DNS lookup time dramatically.

Conclusion

The **centralized DNS architecture** is responsible for much of the 800 ms delay because it creates a bottleneck and forces global users to depend on one server.

A better solution is a **distributed Anycast DNS architecture with multiple redundant DNS servers, optimized TTL values, and effective DNS caching**. Anycast reduces geographical latency, caching reduces repeated queries, and suitable TTL values balance performance with the need for DNS updates.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the **average order acknowledgment latency increases from 1 ms to 40 ms**, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

1. Analyze three possible TCP-related factors responsible for the latency increase, considering **flow control**, **Nagle's algorithm**, and **SYN queue limitations**.
2. Explain how each factor affects transaction processing in high-frequency systems
3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

1. Three TCP-related factors responsible for latency increase

The increase in acknowledgment latency from **1 ms to 40 ms** can be caused by several TCP mechanisms. The three important factors in this case are **flow control, Nagle's algorithm, and SYN queue limitations**.

A. TCP Flow Control

TCP uses a **receive window (rwnd)** to control how much data the sender can transmit before receiving acknowledgments.

During market opening hours, thousands of transactions can generate a large amount of traffic. If the receiving application cannot process data quickly enough, its receive buffer may become full. TCP then reduces the available receive window, causing the sender to slow down.

Effect on trading:

- Data transmission becomes slower.
- Packets may wait in buffers.
- Order acknowledgments are delayed.
- Excessive buffering can increase latency.

Example:

High order traffic → Receive buffer fills → TCP window decreases → Sender slows down → Higher acknowledgment latency

B. Nagle's Algorithm

Nagle's algorithm is designed to reduce network overhead by combining small TCP packets into larger packets before sending them.

This is useful for normal applications, but it can be harmful to **high-frequency trading**, where messages are often very small and must be delivered immediately.

Nagle's algorithm may wait briefly for additional data before transmitting a small packet.

Effect on trading:

- Small order messages may be delayed.

- Trade acknowledgments may wait in the TCP buffer.
- Even a small delay can be significant in high-frequency trading.
- It can contribute to the increase from 1 ms to 40 ms latency.

For latency-sensitive applications, **TCP_NODELAY** can be enabled to disable Nagle's algorithm.

C. SYN Queue Limitations

Before a TCP connection can transfer data, it performs the **three-way handshake**:

SYN → SYN-ACK → ACK

During busy periods, many clients may attempt to establish connections simultaneously. The server maintains a **SYN queue** for connections that are in the handshake stage.

If the queue becomes full:

- New connection requests may be dropped.
- Clients retransmit SYN packets.
- Connection establishment takes longer.
- Application requests are delayed.

This is especially problematic if the trading system frequently creates new TCP connections during market opening.

2. Effect on high-frequency transaction processing

TCP Factor	What happens	Effect on trading
Flow Control	Receive window becomes smaller when buffers fill	Slower order transmission and acknowledgments
Nagle's Algorithm	Small packets may be temporarily combined	Adds delay to small order messages

SYN Queue Limitations	Too many connection requests fill the SYN queue	Connection establishment becomes slow or fails temporarily
------------------------------	---	--

In a high-frequency trading system, **milliseconds matter**. A delay of 40 ms can cause an order acknowledgment to arrive significantly later than expected, potentially affecting trading decisions and customer experience.

3. Recommended TCP configuration changes

A. Disable Nagle's algorithm

Enable **TCP_NODELAY** for latency-sensitive trading connections.

This allows small packets to be sent immediately instead of waiting for additional data.

Recommended:

TCP_NODELAY = ON

This is one of the most direct changes for reducing application-level TCP latency.

B. Optimize TCP receive and send buffers

Configure appropriate **TCP send and receive buffer sizes**.

Buffers should be large enough to handle bursts of trading traffic but not unnecessarily large, because excessive buffering can increase latency.

Use:

- Appropriate **SO_RCVBUF**
 - Appropriate **SO_SNDBUF**
 - Proper application-level queue management
-

C. Increase SYN queue capacity

Increase the server's ability to handle simultaneous connection requests.

Possible measures include:

- Increase the TCP SYN backlog/queue.
- Enable **SYN cookies** as a protection mechanism against SYN floods.
- Use connection pooling so that applications do not repeatedly create new TCP connections.

Connection pooling is particularly useful because an established TCP connection can be reused for multiple orders.

D. Reduce unnecessary connection establishment

Instead of creating a new TCP connection for every transaction:

Less efficient:

Order → TCP handshake → Send order → Close connection

Better:

TCP connection established once → Order 1 → Order 2 → Order 3 → ...

Persistent connections reduce the overhead of repeated TCP handshakes.

E. Monitor retransmissions and congestion

Since the network engineers have already observed increased TCP retransmissions, they should investigate:

- Packet loss
- Network congestion
- Receive-window limitations
- Interface errors
- TCP retransmission timeout (RTO)

Simply changing TCP parameters will not solve the problem if the underlying network is dropping packets.

Conclusion

The increase in order acknowledgment latency from **1 ms to 40 ms** can be strongly associated with **TCP flow-control limitations, Nagle's algorithm, and SYN queue congestion.**

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand